

```
import numpy as np
import matplotlib.pyplot as plt
from collections import deque
```

(a)

```
def load_data(filename):
    graph = {}
    edges = []
    nodes = set()

    with open(filename, "r") as file:
        for line in file:
            if line.startswith("#"):
                continue
            node1, node2 = map(int, line.strip().split())

            nodes.update([node1, node2])
            edges.append((node1, node2))

            if node1 not in graph:
                graph[node1] = set()
            if node2 not in graph:
                graph[node2] = set()

            graph[node1].add(node2)
            graph[node2].add(node1)

    return graph, edges, sorted(nodes)

def create_adjacency_matrix(graph, nodes):
    n = len(nodes)
    node_index = {node: i for i, node in enumerate(nodes)}
    matrix = np.zeros((n, n), dtype=int)

    for node in graph:
        for neighbor in graph[node]:
            matrix[node_index[node]][node_index[neighbor]] = 1

    return matrix

graph, edges, nodes = load_data("ca-GrQc.txt")
# graph stores the 'adjacency list'
# edge stores the 'edge list'

print("\n Edge List (Number of edges: {}):".format(len(edges)))
print("-" * 25)
for i, edge in enumerate(edges[:50]): # Print first 50 edges only for ease of reading
    print(f" {i+1:02}. Node {edge[0]:<4} ↔ Node {edge[1]:<4}")
if len(edges) > 50:
    print(" ... (showing first 50 edges)")

matrix = create_adjacency_matrix(graph, nodes) # adjacency_matrix

print("\n Adjacency Matrix (Size: {} x {}):".format(len(nodes), len(nodes)))
print("-" * (5 * len(nodes) + 6))

print("      " + " ".join(f"{node:3}" for node in nodes))

print("-" * (5 * len(nodes) + 6))

for i, row in enumerate(matrix):
    print(f"{nodes[i]:3} | " + " ".join(f"{val:3}" for val in row))

print("-" * (5 * len(nodes) + 6))
```



```
Edge List (Number of edges: 28980):
-----
01. Node 3466 ↔ Node 937
02. Node 3466 ↔ Node 5233
03. Node 3466 ↔ Node 8579
04. Node 3466 ↔ Node 10310
05. Node 3466 ↔ Node 15931
```

```

06. Node 3466 ↔ Node 17038
07. Node 3466 ↔ Node 18720
08. Node 3466 ↔ Node 19607
09. Node 10310 ↔ Node 1854
10. Node 10310 ↔ Node 3466
11. Node 10310 ↔ Node 4583
12. Node 10310 ↔ Node 5233
13. Node 10310 ↔ Node 9572
14. Node 10310 ↔ Node 10841
15. Node 10310 ↔ Node 13056
16. Node 10310 ↔ Node 14982
17. Node 10310 ↔ Node 16310
18. Node 10310 ↔ Node 19640
19. Node 10310 ↔ Node 23855
20. Node 10310 ↔ Node 24372
21. Node 10310 ↔ Node 24814
22. Node 5052 ↔ Node 899
23. Node 5052 ↔ Node 1796
24. Node 5052 ↔ Node 2287
25. Node 5052 ↔ Node 3096
26. Node 5052 ↔ Node 3386
27. Node 5052 ↔ Node 4472
28. Node 5052 ↔ Node 5346
29. Node 5052 ↔ Node 5740
30. Node 5052 ↔ Node 6094
31. Node 5052 ↔ Node 6376
32. Node 5052 ↔ Node 9124
33. Node 5052 ↔ Node 10235
34. Node 5052 ↔ Node 10427
35. Node 5052 ↔ Node 10597
36. Node 5052 ↔ Node 15159
37. Node 5052 ↔ Node 16148
38. Node 5052 ↔ Node 16741
39. Node 5052 ↔ Node 18235
40. Node 5052 ↔ Node 18549
41. Node 5052 ↔ Node 19297
42. Node 5052 ↔ Node 20511
43. Node 5052 ↔ Node 20595
44. Node 5052 ↔ Node 20613
45. Node 5052 ↔ Node 24371
46. Node 5052 ↔ Node 24559
47. Node 5052 ↔ Node 24731
48. Node 5052 ↔ Node 25102
49. Node 5052 ↔ Node 25271
50. Node 5052 ↔ Node 25396
... (showing first 50 edges)

```

Adjacency Matrix (Size: 5242 x 5242):

(b)

Image visualized using Cytoscape and saved as "ca-GrQc.png"

(c)

```

def calculate_sparseness(graph, edges):
    N = len(graph)
    E = len(edges)

    if N <= 1:
        return 1.0

    max_edges = (N * (N - 1)) / 2 # Edges in a complete graph
    sparseness = 1 - (E / max_edges)

    return sparseness

```

```

sparseness = calculate_sparseness(graph, edges)
print(f"Sparseness of the network: {sparseness:.5f}")

```

→ Sparseness of the network: 0.99789

High sparseness (value close to 1) means the network is sparse. Since N is large and $E \ll N(N-1)/2$, this network is highly sparse. This may be due to most nodes being only locally connected, leading to a very low density.

(d)

```
def calculate_average_degree(graph, edges):
    N = len(graph)
    E = len(edges)

    if N == 0:
        return 0.0

    avg_degree = (2 * E) / N
    return avg_degree
```

```
average_degree = calculate_average_degree(graph, edges)
print(f"Average Degree <k>: {average_degree:.6f}")
```

→ Average Degree <k>: 11.056849

(e)

```
def calculate_degree_distribution(graph):
    degree_count = {}

    for node in graph:
        degree = len(graph[node])
        degree_count[degree] = degree_count.get(degree, 0) + 1

    total_nodes = len(graph)
    p_k = {k: v / total_nodes for k, v in degree_count.items()}

    return p_k

def plot_degree_distribution(graph):
    p_k = calculate_degree_distribution(graph)

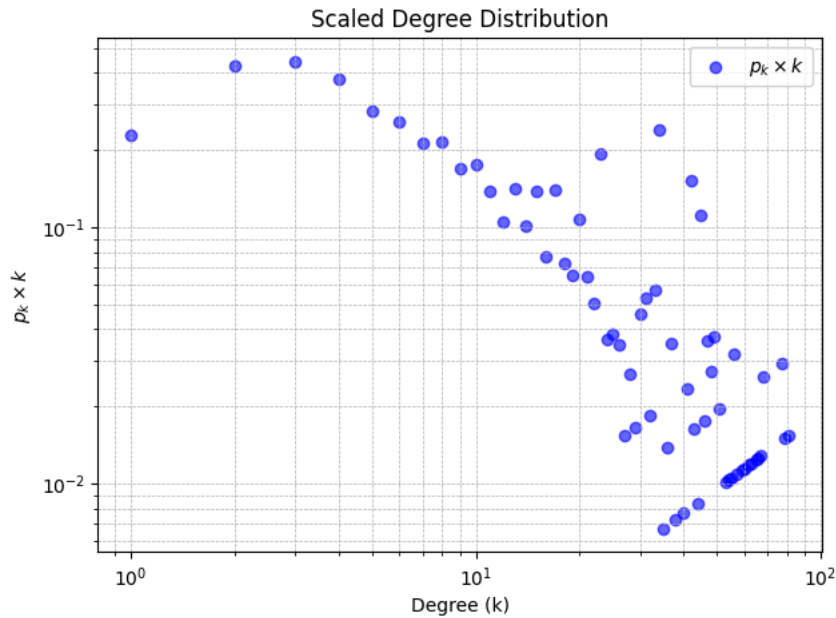
    degrees = np.array(list(p_k.keys()))
    probabilities = np.array(list(p_k.values()))

    plt.figure(figsize=(7, 5))
    plt.scatter(degrees, probabilities * degrees, color='blue', alpha=0.6, label=r"$p_k \times k$")
    plt.xscale('log')
    plt.yscale('log')
    plt.xlabel("Degree (k)")
    plt.ylabel(r"$p_k \times k$")
    plt.title("Scaled Degree Distribution")
    plt.legend()
    plt.grid(True, which="both", linestyle="--", linewidth=0.5)

    plt.savefig("degree_distribution.png", dpi=300)
    print("Saved as degree_distribution.png")
```

```
plot_degree_distribution(graph)
```

↳ Saved as degree_distribution.png



(f)

```
def shortest_path_bfs(graph, start):
    queue = deque([start])
    distances = {start: 0}

    while queue:
        node = queue.popleft()
        for neighbor in graph[node]:
            if neighbor not in distances:
                distances[neighbor] = distances[node] + 1
                queue.append(neighbor)

    return distances

def calculate_average_path_lengths(graph):
    avg_path_lengths = []

    for node in graph:
        shortest_paths = shortest_path_bfs(graph, node)
        avg_length = sum(shortest_paths.values()) / len(shortest_paths)
        avg_path_lengths.append(avg_length)

    return avg_path_lengths

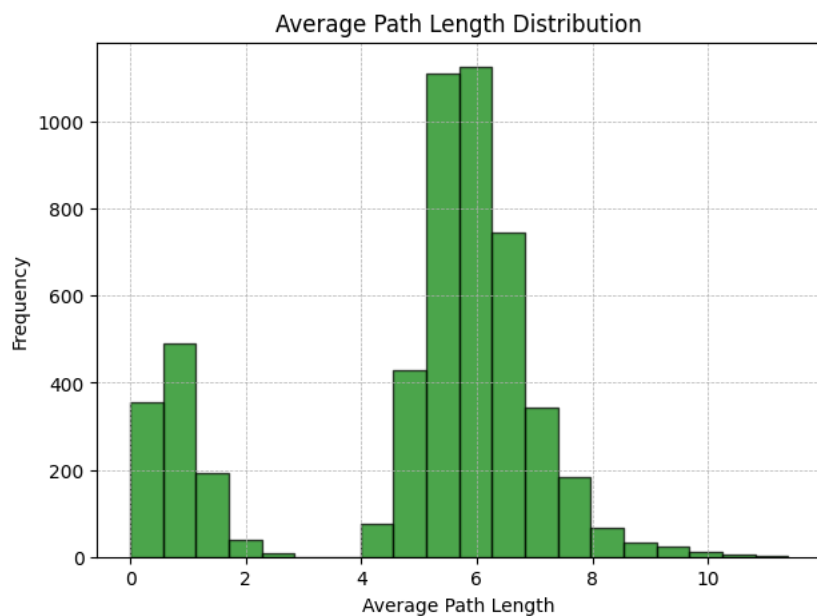
def plot_avg_path_length_distribution(graph):
    avg_lengths = calculate_average_path_lengths(graph)

    plt.figure(figsize=(7, 5))
    plt.hist(avg_lengths, bins=20, color='green', alpha=0.7, edgecolor='black')
    plt.xlabel("Average Path Length")
    plt.ylabel("Frequency")
    plt.title("Average Path Length Distribution")
    plt.grid(True, linestyle="--", linewidth=0.5)

    plt.savefig("path_length_distribution.png", dpi=300)
    print("Saved as path_length_distribution.png")

plot_avg_path_length_distribution(graph)
```

↳ Saved as path_length_distribution.png



(g)

```
def calculate_clustering_coefficients(graph):
    clustering_coeffs = {}

    for node in graph:
        neighbors = list(graph[node])
        k = len(neighbors)

        if k < 2:
            clustering_coeffs[node] = 0
            continue

        actual_edges = 0
        for i in range(k):
            for j in range(i + 1, k):
                if neighbors[j] in graph[neighbors[i]]:
                    actual_edges += 1

        max_edges = k * (k - 1) / 2
        clustering_coeffs[node] = actual_edges / max_edges if max_edges > 0 else 0

    return clustering_coeffs

def calculate_avg_clustering_by_degree(graph):
    clustering_coeffs = calculate_clustering_coefficients(graph)

    degree_clustering = {}
    degree_count = {}

    for node, C in clustering_coeffs.items():
        k = len(graph[node])
        if k not in degree_clustering:
            degree_clustering[k] = 0
            degree_count[k] = 0

        degree_clustering[k] += C
        degree_count[k] += 1

    avg_C_k = {k: degree_clustering[k] / degree_count[k] for k in degree_clustering}

    return avg_C_k

def plot_clustering_coefficient(graph):
    avg_C_k = calculate_avg_clustering_by_degree(graph)

    degrees = np.array(list(avg_C_k.keys()))
    clustering_values = np.array(list(avg_C_k.values()))
```

```

plt.figure(figsize=(7, 5))
plt.scatter(degrees, clustering_values, color='red', alpha=0.6, label=r"$C(k) \times k$")
plt.xscale('log')
plt.yscale('log')
plt.xlabel("Degree (k)")
plt.ylabel(r"Clustering Coefficient $C(k)$")
plt.title("Clustering Coefficient vs Degree")
plt.legend()
plt.grid(True, which="both", linestyle="--", linewidth=0.5)

plt.savefig("clustering_coefficient.png", dpi=300)
print("Saved as clustering_coefficient.png")

```

plot_clustering_coefficient(graph)

 Saved as clustering_coefficient.png

